

127. La pesanteur et le saut

Tomber, et retomber.

1. Ce qu'il faut comprendre

Le processeur ne connaît **pas les nombres négatifs** : `0 - 1` vaut 255. Une vitesse qui doit monter *et* descendre ne peut donc pas s'écrire directement.

On la décale : `ZEROV` vaut 16, et la vitesse vit autour. **Moins de 16, on monte ; plus de 16, on tombe.** Sans ce décalage, un saut deviendrait une chute de 240 pixels par image.

`const uint8_t ZEROV = 16;` ne coûte rien : la valeur est connue à la compilation, et le compilateur l'écrit directement dans le code. Un `const` n'occupe pas d'octet de mémoire.

La pesanteur ajoute 1 à chaque image. Sauter, c'est poser une vitesse bien en dessous de 16 : elle remonte d'elle-même, le mouvement s'inverse, et la courbe est celle d'un vrai saut.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **618 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage — la mise en route de la console
main()	176 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	67 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	144 o	← CopierTuiles — 8 lettres sur 43, et 1 tuile dessinée
DonneesTransfert	10 o	← InstallerTransfert — la petite routine qui copie les lutins
"A POUR SAUTER"	13 o	← texte() — un texte gravé

Lettres gravées : AEOPRSTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

`sprite() × 1` 23 o sur place, dans `main()`

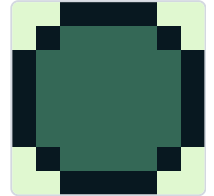
```
bouton() × 1    12 o sur place, dans main() + LireManette (37 o)
texte() × 1     11 o sur place, dans main() + EcrireTexte (10 o)
image() × 1      6 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <Tuile>      // un dessin de 8 × 8 pixels
#include <texte>       // écrit un texte à l'écran
#include <bouton>      // lit un bouton de la manette
#include <sprite>      // place un lutin de 8 × 8 au pixel près
```

2. UN DESSIN — BALLE

```
Tuile BALLE = {
    "00333300",
    "03222230",
    "32222223",
    "32222223",
    "32222223",
    "32222223",
    "32222223",
    "03222230",
    "00333300",
};
```



à l'écran

3. UNE CONSTANTE — ZEROV

```
const uint8_t ZEROV = 16;
```

4. UNE CONSTANTE — SOL

```
const uint8_t SOL = 120;
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    uint8_t x = 76;
    uint8_t y = SOL;
    uint8_t vy = ZEROV;
    uint8_t auSol = 1;
    uint8_t aAvant = 0;

    texte(4, 2, "A POUR SAUTER");

    while (true) {
        image();

        uint8_t a = bouton(A);

        if (a && !aAvant && auSol) {
            vy = ZEROV - 7;
            auSol = 0;
        }

        aAvant = a;

        vy++;
        if (vy > ZEROV + 4) vy = ZEROV + 4;

        if (vy < ZEROV) y = y - ZEROV + vy;
        if (vy > ZEROV) y = y + vy - ZEROV;

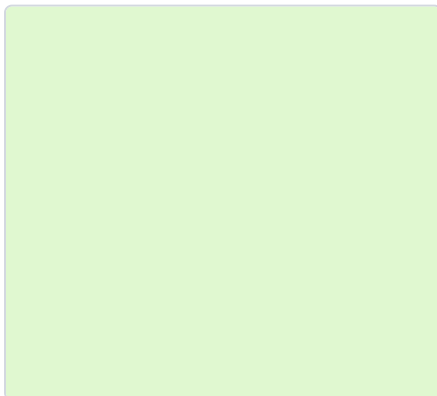
        if (y >= SOL) {
            y = SOL;
            vy = ZEROV;
            auSol = 1;
        }

        sprite(0, x, y, BALLE);
    }

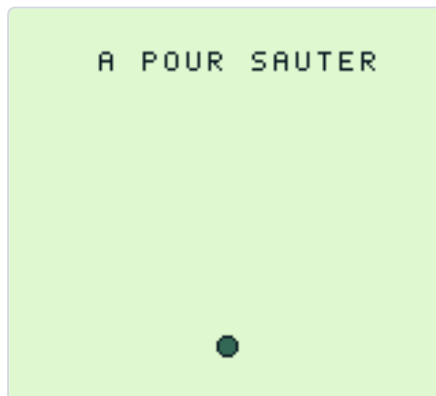
    return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran n'a pas bougé

5. Ce qu'on doit voir

La boule saute, ralentit en l'air, et retombe sur le sol.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ elle part posée au sol
- ✓ A la fait monter (120 vers 99)
- ✓ de plusieurs cases (21 pixels)
- ✓ et elle retombe exactement au sol

Ensuite : **128. Le verre qui se brise** — chapitre 23 : Un vrai jeu. · gameboy3 — écrire une cartouche Game Boy en C++